PROJECT REPORT

EXPERIMENT – 07

Project title:

MQTT-Based Remote HVAC Monitoring System

Submitted by:

Arundeepak S -24MER004

Deepak P -24MER006

Dharanidharan R S -24MER007

Dinesh k -24MER009

Vishnuvarthan P -24MER063

1. Project Overview

This project implements a remote HVAC (Heating, Ventilation, and Air Conditioning) monitoring system built around an ESP8266 NodeMCU microcontroller. The system continuously measures ambient temperature and humidity using a DHT sensor, displays live readings and status on a 16x2 I2C LCD, raises a local audible alert when readings cross safe operating thresholds, and publishes the sensor data to the ThingSpeak cloud platform over MQTT for remote monitoring, logging, and analysis.

The solution is designed as a low-cost, easily reproducible IoT monitoring node suitable for server rooms, storage facilities, or any enclosed space where temperature and humidity need to stay within a controlled range.

2. Problem Statement

Traditional HVAC systems in server rooms, warehouses, and equipment enclosures are often monitored manually or not monitored at all, which delays the detection of temperature or humidity excursions that can damage sensitive equipment, spoil stored materials, or create unsafe environments. Facility staff frequently have no way to check conditions remotely, and by the time an issue is noticed on-site, damage may already have occurred. There is a need for a low-cost, always-on system that can measure environmental conditions continuously, alert on-site personnel immediately, and make the data available remotely so that HVAC conditions can be tracked and analyzed from anywhere.

3. Proposed Solution

An ESP8266 NodeMCU-based monitoring node is proposed that:

- Reads temperature and humidity at regular intervals using a DHT sensor.
- Displays the live readings and a NORMAL/ALERT status on a local I2C LCD.
- Triggers a buzzer immediately when temperature or humidity exceeds pre-configured safe thresholds.
- Connects to Wi-Fi and publishes readings to a ThingSpeak channel over the MQTT protocol at fixed intervals.
- Allows facility staff to view historical and live HVAC data remotely through the ThingSpeak dashboard.

This approach removes the need for manual checks, provides instant local alerting, and gives remote visibility into HVAC conditions through the cloud.

4. System Architecture

The system architecture consists of three layers:

- Sensing & Actuation Layer: DHT sensor for environmental data acquisition, buzzer for local alerting, and I2C LCD for on-site display.
- Processing & Connectivity Layer: ESP8266 NodeMCU reads sensor data, evaluates threshold conditions, drives the display and buzzer, and manages the Wi-Fi/MQTT connection.
- Cloud Layer: ThingSpeak MQTT broker receives published readings on channel fields (temperature, humidity, alert status), stores them, and makes them available for visualization, charting, and analysis.

Data flow: Sensor → NodeMCU (threshold check) → LCD/Buzzer (local feedback) in parallel with NodeMCU → Wi-Fi → MQTT broker → ThingSpeak Channel → Remote dashboard.

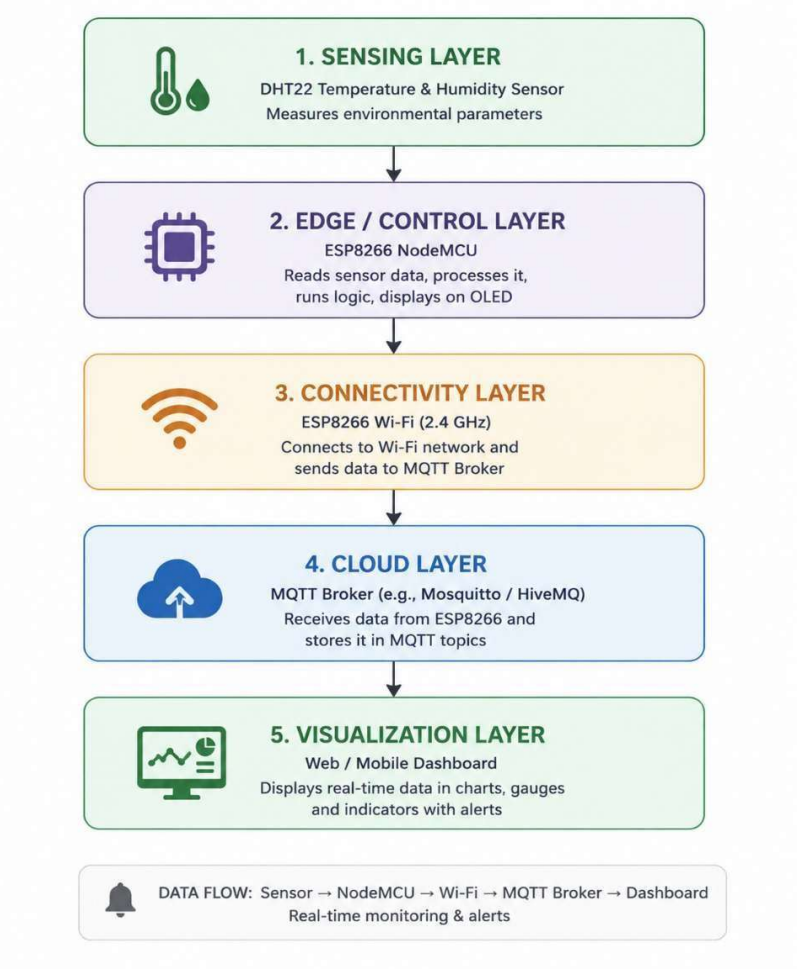

Figure 1: system architecture

5. Circuit Table

Bill of components used to wire the circuit:

Component	Quantity
ESP8266 NodeMCU	1
DHT22 Sensor	1
OLED/LCD Display (I2C)	1
Breadboard	1
Jumper Wires	1 Set

6. Circuit Design

The DHT sensor's data pin connects to a GPIO on the NodeMCU, while the I2C LCD (via a PCF8574 backpack) connects to the NodeMCU's SDA and SCL lines, sharing power and ground on the breadboard. The wiring layout is shown below.

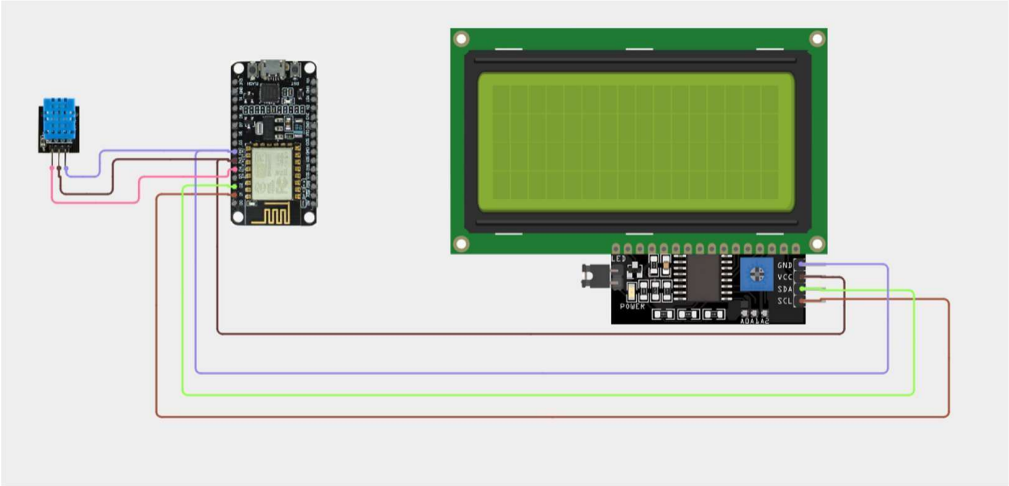

Figure 1: Circuit Design – DHT Sensor and I2C LCD Wiring with ESP8266 NodeMCU

7. Algorithm

- Step 1: Initialize serial communication, buzzer pin, DHT sensor, and I2C LCD.
- Step 2: Display a startup message and connect to the configured Wi-Fi network.
- Step 3: Connect to the ThingSpeak MQTT broker using the channel's client ID, username, and password; retry every 5 seconds on failure.
- Step 4: In the main loop, verify the MQTT connection is alive; reconnect if it has dropped.
- Step 5: Read temperature and humidity from the DHT sensor; if the reading is invalid, show a sensor error on the LCD and retry.
- Step 6: Evaluate alert conditions — temperature above the high threshold, or humidity outside the safe low/high band.
- Step 7: Update the LCD with the current readings and a NORMAL or ALERT status.
- Step 8: Drive the buzzer HIGH if any alert condition is active, otherwise keep it LOW.
- Step 9: Print the readings and status to the serial monitor for debugging.
- Step 10: Every publish interval (20 seconds), publish temperature, humidity, and alert flag to the ThingSpeak channel over MQTT.
- Step 11: Wait briefly and repeat from Step 4.

8. Coding

The complete Arduino sketch (ESP8266/ESP32, DHT sensor, I2C LCD, buzzer, ThingSpeak MQTT publish) used for this experiment is listed below.

```
#include <WiFi.h>

#include <PubSubClient.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include "DHT.h"


// ---------- WiFi ----------

const char* ssid = "Vishnuvarthan";

const char* password = "vishnu2007";


// ---------- ThingSpeak MQTT ----------

const char* mqttServer = "mqtt3.thingspeak.com";

const int mqttPort = 1883;

unsigned long channelID = 3461751;              // your channel ID


const char* mqttClientID = "PRQUOzUeCA0EIRgEIwYHEQM";

const char* mqttUsername = "PRQUOzUeCA0EIRgEIwYHEQM";

const char* mqttPassword = "nijosWkKn7RfQgBXP736aW1T";


WiFiClient espClient;

PubSubClient client(espClient);


// ---------- DHT11 ----------

#define DHTPIN 4

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
```

```
// ---------- Buzzer ----------
#define BUZZER 5

// ---------- LCD (I2C) ----------
#define SDA_PIN 21
#define SCL_PIN 22
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---------- Thresholds ----------
const float TEMP_THRESHOLD = 30.0;
const float HUMID_LOW = 30.0;
const float HUMID_HIGH = 70.0;

unsigned long lastPublish = 0;
const unsigned long publishInterval = 20000;

void connectWiFi() {
  Serial.println("Connecting to WiFi...");
  lcd.setCursor(0, 1);
  lcd.print("Connecting WiFi");

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected!");
  Serial.println(WiFi.localIP());

  lcd.clear();
```

```
  lcd.setCursor(0, 0);
  lcd.print("WiFi Connected!");
  delay(1500);
  lcd.clear();
}

void connectMQTT() {
  while (!client.connected()) {
    Serial.println("Connecting to ThingSpeak MQTT broker...");
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Connecting MQTT");

    if (client.connect(mqttClientID, mqttUsername, mqttPassword)) {
      Serial.println("MQTT Connected!");
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("MQTT Connected!");
      delay(1000);
      lcd.clear();
    } else {
      Serial.print("MQTT connection failed, rc=");
      Serial.print(client.state());
      Serial.println(" retrying in 5 seconds");
      lcd.setCursor(0, 1);
      lcd.print("MQTT Failed");
      delay(5000);
    }
  }
}
```

```
void setup() {
  Serial.begin(115200);

  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);

  dht.begin();

  Wire.begin(SDA_PIN, SCL_PIN);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Fridge Monitor");

  connectWiFi();
  client.setServer(mqttServer, mqttPort);
  connectMQTT();

  lcd.clear();
}

void loop() {
  if (!client.connected()) {
    connectMQTT();
  }
  client.loop();

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
```

```
if (isnan(humidity) || isnan(temperature)) {
  Serial.println("DHT11 ERROR");
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Sensor Error!");
  lcd.setCursor(0, 1);
  lcd.print("Check DHT wiring");
  delay(2000);
  return;
}

bool alert = (temperature > TEMP_THRESHOLD) || (humidity < HUMID_LOW) || (humidity > HUMID_HIGH);

// ---------- LCD Display ----------
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("T:");
lcd.print(temperature, 1);
lcd.print("C H:");
lcd.print(humidity, 1);
lcd.print("%");

lcd.setCursor(0, 1);
lcd.print(alert ? "STATUS: ALERT!" : "STATUS: NORMAL");

// ---------- Buzzer ----------
digitalWrite(BUZZER, alert ? HIGH : LOW);

// ---------- Serial Debug ----------
```

```
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" C");
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");
  Serial.println(alert ? "ALERT!" : "Normal");

  // ---------- MQTT Publish ----------
  if (millis() - lastPublish > publishInterval) {
    String topic = "channels/" + String(channelID) + "/publish";
    String payload = "field1=" + String(temperature) + "&field2=" + String(humidity) + "&field3=" + String(alert ? 1
: 0);

    if (client.publish(topic.c_str(), payload.c_str())) {
      Serial.println("MQTT Publish successful");
    } else {
      Serial.println("MQTT Publish failed");
    }
    lastPublish = millis();
  }

  delay(2000);
}
```

9. System Working

On power-up, the NodeMCU initializes its peripherals and displays a startup message on the LCD while it connects to Wi-Fi and then to the ThingSpeak MQTT broker, showing connection status at each stage. Once connected, the system enters its main loop: every 2 seconds it reads temperature and humidity from the DHT sensor and updates the LCD with the live values and a NORMAL/ALERT status line. If the temperature exceeds the high threshold, or the humidity falls outside the configured safe band, the buzzer is switched on immediately to give an audible local warning; it switches off again once conditions return to normal. In parallel, every 20 seconds the current temperature, humidity, and alert flag are packaged and published to the ThingSpeak channel over MQTT, so the same data that is shown locally on the LCD becomes visible on the remote ThingSpeak dashboard for historical trend analysis. If the MQTT connection drops at any point, the system automatically attempts to reconnect every 5 seconds without needing a manual reset.

The serial monitor output below confirms this behaviour under both alert and normal operating conditions.

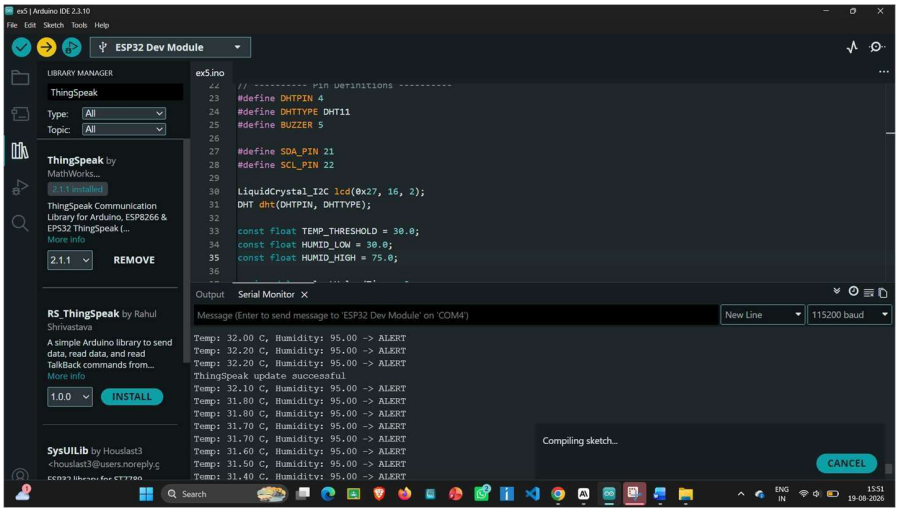

Figure 2: Serial Monitor Output Showing Alert Condition (Temp: 32°C, Humidity: 95% → ALERT)

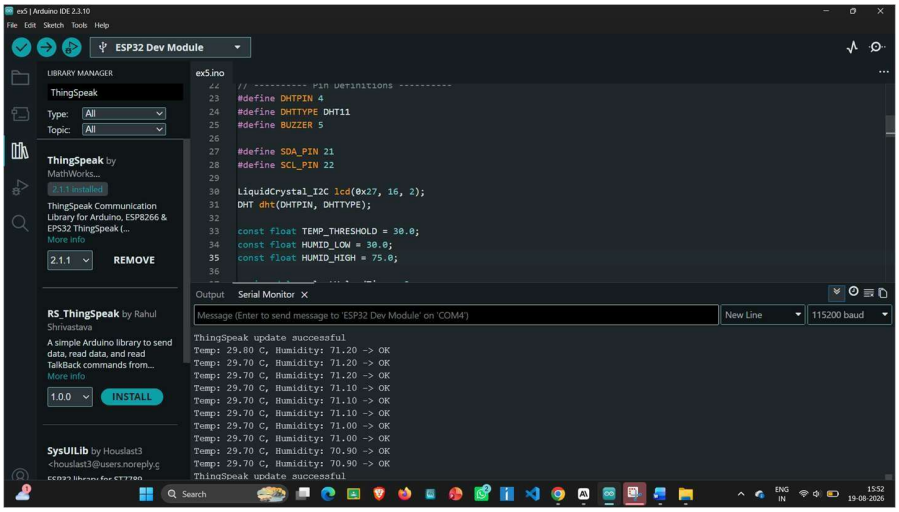

Figure 3: Serial Monitor Output Showing Normal Condition (Temp: 29.7°C, Humidity: 71% → OK)

10. Bill of Materials

S.No	Component	Quantity
1	ESP8266 NodeMCU	1
2	DHT22 Sensor	1
3	OLED/LCD Display (I2C)	1
4	Breadboard	1
5	Jumper Wires	1 Set

11. Prototype Implementation

The prototype was assembled on a breadboard with the DHT sensor, I2C LCD, and buzzer wired to the ESP8266 NodeMCU as per the circuit design. Firmware was uploaded using the Arduino IDE, and the device was verified to connect to the local Wi-Fi network and the ThingSpeak MQTT broker on power-up, after which live readings and status were confirmed on both the LCD and the serial monitor.

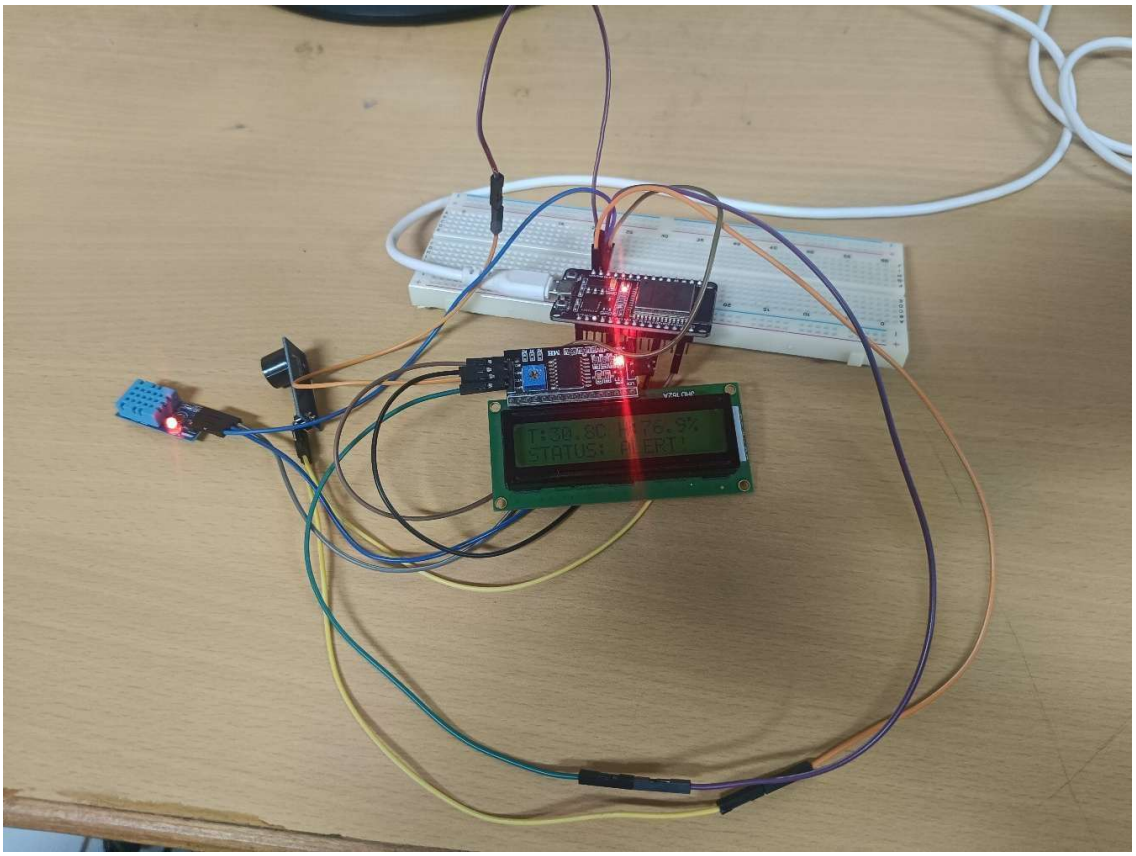

Figure 4: Assembled hardware prototype on breadboard

12. Results & Performance Analysis

The device was registered on ThingSpeak as an MQTT device authorized to publish and subscribe to the HVAC monitoring channel, confirming successful cloud connectivity.

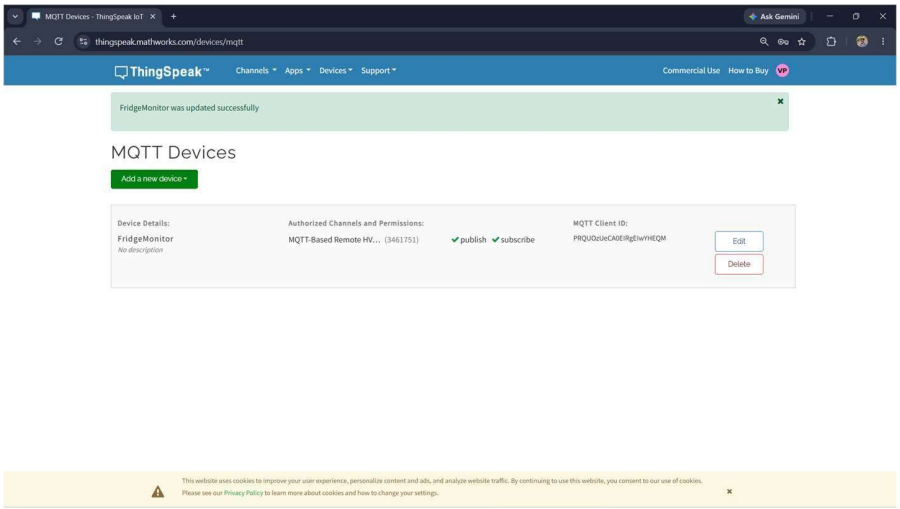

Figure 4: ThingSpeak MQTT Device Configuration (FridgeMonitor Device, Channel 3461751)

The live ThingSpeak dashboard displays Field 1 (Temperature) and Field 2 (Humidity) charts along with a Field 3 lamp indicator that visually reflects the alert status. The figures below show the dashboard during an alert condition (indicator lit) and after conditions returned to normal (indicator off).

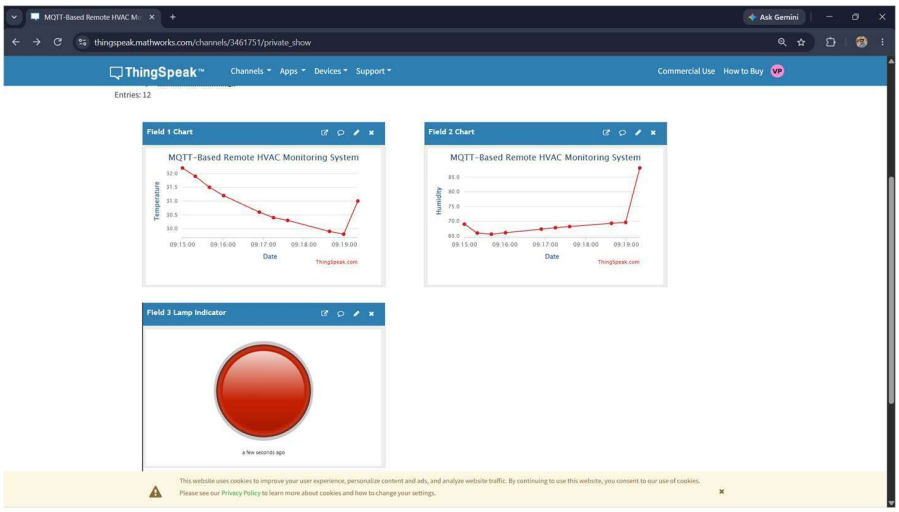

Figure 5: ThingSpeak Dashboard – Temperature and Humidity Charts with Alert Indicator ON

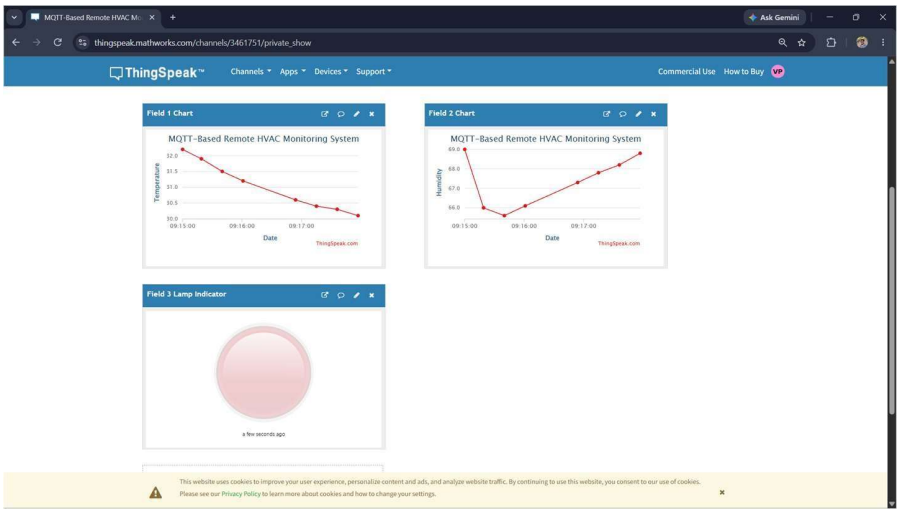

Figure 6: ThingSpeak Dashboard – Temperature and Humidity Charts with Alert Indicator OFF (Normal State)

The system was tested over multiple operating cycles with the following observations:

- Sensor readings updated reliably every 2 seconds and were correctly reflected on the LCD.
- The buzzer activated immediately when temperature exceeded 30°C or humidity moved outside the 30–70% band, and deactivated as soon as conditions returned to normal.
- MQTT publishes to the ThingSpeak channel succeeded consistently at the configured 20-second interval, with automatic reconnection handling observed Wi-Fi/MQTT drops without requiring manual intervention.
- Remote data was visible on the ThingSpeak dashboard with a delay of only a few seconds after each publish, confirming the system's suitability for near real-time remote HVAC monitoring.

Overall, the prototype met its design goals: reliable local sensing and alerting combined with dependable remote data logging, at low hardware cost and with minimal power/connectivity overhead.